

# Atividade Prática - Representação do Mundo

**Disciplina:** Inteligência Artificial para Jogos I

**Aluno:** [PREENCHER]

**Link da solução:** <https://github.com/SEU-USUARIO/representacao-mundo>

## 1. Objetivo

Foi desenvolvido um nível de jogo 2D com um cenário dividido em nove áreas organizadas em uma malha 3x3. O foco da solução é demonstrar uma representação espacial simples que permita limitar a atualização dos NPCs às regiões relevantes para o jogador.

## 2. Representação do mundo

Cada célula da malha é representada por uma estrutura **Area**, que armazena sua posição na malha, seu estado ativo/inativo, os NPCs e os itens coletáveis presentes naquela região. As entidades utilizam coordenadas globais do mundo; a malha é usada para organizar essas entidades espacialmente.

## 3. Viewport independente

A visualização utiliza uma **Camera2D** independente da malha. A câmera acompanha o jogador, mas as posições do jogador, NPCs e itens continuam armazenadas em coordenadas absolutas do mundo. Assim, a parte do cenário mostrada na tela não determina quais células existem ou onde as entidades realmente estão.

## 4. Ativação das áreas

A área atual do jogador permanece sempre ativa. A posição local do jogador dentro da célula é usada para verificar sua proximidade das bordas. Ao se aproximar de uma borda, a área vizinha correspondente também é ativada. Próximo a um canto, a área diagonal é ativada. Essa estratégia mantém no máximo quatro áreas ativas simultaneamente.

## 5. Atualização seletiva dos NPCs

Somente NPCs armazenados em áreas ativas executam sua rotina de atualização. NPCs em regiões inativas continuam existindo no mundo, mas não recebem atualização naquele frame. O modo de debug exibe tanto a quantidade total de NPCs quanto a quantidade que realmente foi atualizada, tornando visível o efeito da subdivisão espacial.

## 6. Comportamento dos NPCs

O comportamento automático escolhido foi perseguição em linha reta. Cada NPC calcula a diferença entre sua posição e a posição do jogador, normaliza esse vetor e se movimenta na direção resultante. Em caso de sobreposição, o jogador recebe dano com um intervalo mínimo entre ataques.

## 7. Mudança de área

NPCs podem atravessar os limites das células. Depois de atualizar um NPC, sua nova linha e coluna são calculadas. Caso a célula tenha mudado, a transferência é registrada e aplicada somente após terminar o processamento do frame. Isso evita modificar o vetor de NPCs enquanto ele está sendo percorrido.

## 8. Itens, combate e condição de término

Itens de vida recuperam saúde e itens de munição adicionam usos ao ataque. O ataque consome uma unidade de munição e causa dano aos NPCs dentro de um raio ao redor do jogador. O jogador perde ao ficar sem vida e vence quando permanece vivo até o final do tempo configurado.

## 9. Testes de balanceamento

A solução possui três cenários rápidos: 10, 50 e 200 NPCs por área, correspondendo inicialmente a 90, 450 e 1800 inimigos. O usuário pode alternar entre eles com as teclas 1, 2 e 3. O painel de debug ajuda a comparar quantos NPCs existem no mundo com quantos estão sendo atualizados em cada frame.

## 10. Organização do código

O projeto foi escrito em C++17 com raylib. A classe **Jogo** controla o loop principal; **Mundo** gerencia a malha, áreas, NPCs e itens; **Jogador** controla movimento, vida e munição; **NPC** contém a automatização; e **Area** agrupa as entidades por região.

## 11. Controles

| Tecla        | Função                                          |
|--------------|-------------------------------------------------|
| WASD / setas | Movimentar o jogador                            |
| Espaço       | Usar munição e atacar NPCs próximos             |
| F1           | Ativar/desativar informações de debug           |
| 1 / 2 / 3    | Trocar a quantidade de NPCs do cenário de teste |
| R            | Reiniciar após vitória ou derrota               |

## 12. Compilação e execução

O projeto utiliza CMake. Se a raylib não estiver instalada no sistema, o CMake tenta baixá-la automaticamente com FetchContent na primeira configuração.

```
cmake -S . -B build
cmake --build build -j
./build/representacao_mundo
```

## 13. Link para a solução

**IMPORTANTE:** antes de entregar, substituir o endereço abaixo pelo link real do repositório ou página onde o projeto foi disponibilizado para teste.

<https://github.com/SEU-USUARIO/representacao-mundo>